Nimbus — High Level Design

Version: 1.5 (living document — expect this to change as we build) **Author:** Sujal Kumar Singh **Last updated:** 2026-08-18

> **New here? Do not start with this file.** `OVERVIEW.md` explains the system in plain English. `ARCHITECTURE.md` draws every part of it. This document is the build reference — exact columns, exact steps, exact endpoints — and assumes you already know what we are building and why.

---

1. What this is

A multi-tenant business email platform, built around a **deduplicating storage engine**.

One-line explanation:

> It's a mail server that stores every attachment once, no matter how many mailboxes receive it.

---

2. Why this problem

Business email providers sell a mailbox for roughly **$2/month** and operate at the scale of millions of mailboxes. At that price, **storage is the dominant cost per mailbox**.

The naive failure case:

- A CEO emails a 25 MB pitch deck to 40 colleagues.
- A naive mail server writes 40 copies → **1 GB stored**.
- Nimbus writes 1 copy + 40 pointers → **25 MB stored**.

At 10 million mailboxes, that difference decides whether the product is profitable.

Where dedup helps and where it does not

	Receiving mail	Sending mail
One 25 MB file to 40 people	Must **store** 1 GB → very avoidable	Must **transmit** 1 GB → unavoidable
Does dedup help?	Yes. It is the storage engine.	No. SMTP carries every byte regardless.

Nimbus is a receive-side system. That is deliberate — it is where dedup is load-bearing.

To save *bandwidth* you must stop attaching and start linking (what Gmail does above 25 MB). That is noted as a possible future feature, not part of v1.

Known trade-off (say this out loud in interviews)

Dedup is not free:

- It trades storage for **random I/O** — a mailbox's data is scattered across shared blobs.
- It creates a **garbage collection problem**.
- Microsoft Exchange *removed* Single Instance Storage in Exchange 2010 for exactly this reason.
- It is worth it again today because object storage is cheap and reads are network-bound, not seek-bound.

---

3. Goals

1. Accept real email over SMTP.
2. Store each unique attachment exactly once.
3. Deliver one message to many mailboxes without duplicating bytes.
4. Search a mailbox fast, with a real query language.
5. Track logical vs physical storage, and reclaim space safely.
6. Let a reseller provision domains and mailboxes through an API.

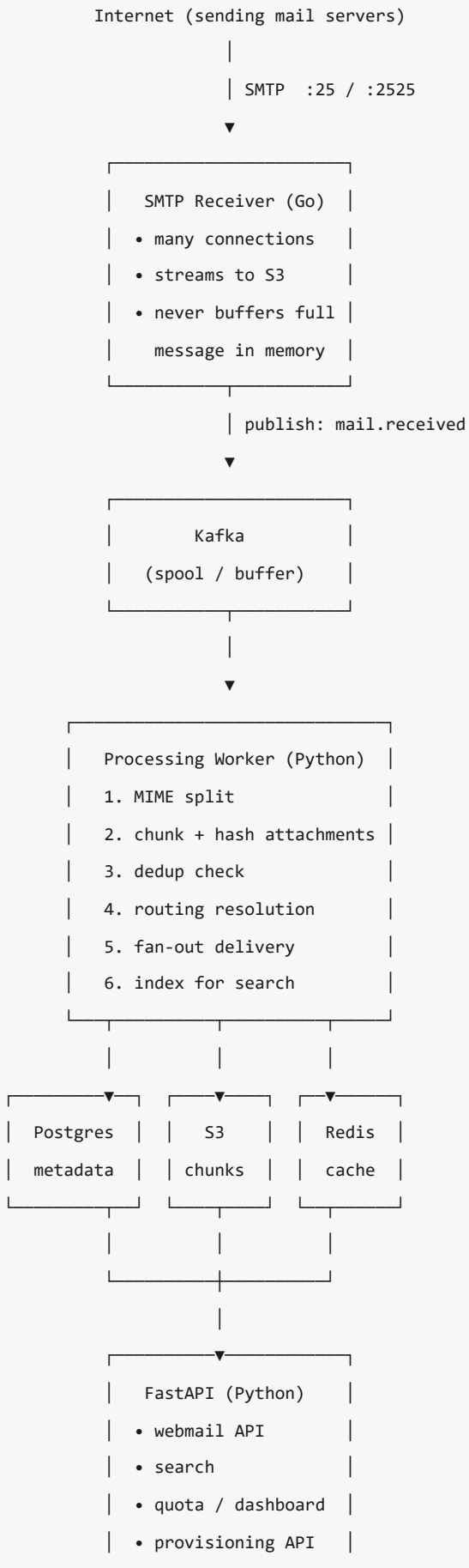
4. Non-goals (v1)

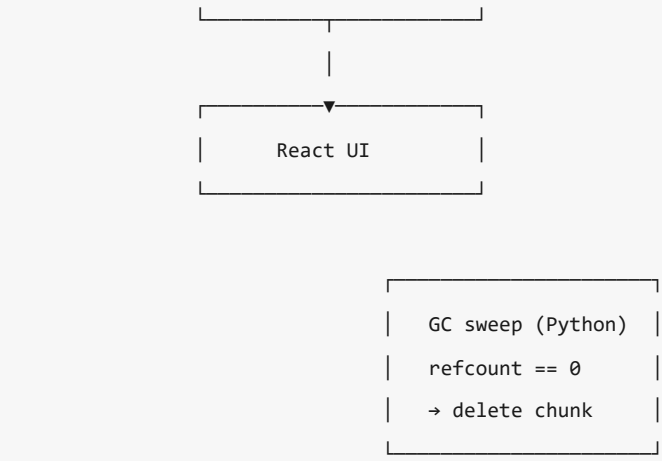
Explicitly out of scope, so scope does not creep:

- Outbound sending (SMTP client, send queue, DKIM signing, bounce handling)
- Send Later / Unsend / Read Receipts — all require the outbound path
- Calendar, contacts, signatures, templates
- IMAP / POP3 server (webmail API only)
- Spam scoring beyond basic SPF/DMARC checks
- Mobile apps
- IMAP migration from another provider *(strong stretch goal)*

---

5. Architecture





There is no snooze worker. Snooze is the predicate `snooze_until > now()`, evaluated at read time (§9.5).

---

6. Components

Component	Language	Responsibility	Why this language
SMTP Receiver	Go	Accept SMTP connections, stream raw message to S3, publish event	Thousands of concurrent connections; 25 MB streams at constant memory. Python's GIL and per-connection cost make this the wrong job for it.
Processing Worker	Python	MIME split, chunk, dedup, route, deliver, index	Rich stdlib (`email`), fast to write, not latency-critical
API	Python / FastAPI	Webmail, search, quota, provisioning, admin	Most of the code lives here. Speed of development matters more than raw throughput.
GC Worker	Python	Find refcount-zero chunks, delete safely	Batch job, latency irrelevant

---

7. Data stores

Store	Holds	Why not somewhere else
Postgres (RDS)	resellers, domains, mailboxes, aliases, messages, blobs, chunk_map, refcounts, snoozes	Needs ACID and joins. Source of truth.
S3 (MinIO local)	attachment chunks, raw message bodies	Blobs must never enter the database. Cheap, unlimited.
Redis (ElastiCache)	valid-address set for `RCPT TO`	Sub-ms lookups. **Not** the blob-existence cache or the dedup lock (dropped in block C, §9.1b), and **not** a snooze timer set (dropped in block H, §9.5) — Redis here is a cache that is rebuilt from Postgres at startup, so nothing may live in it that cannot be rebuilt. Unread counts and rate limits are listed nowhere else and are not built.
Kafka (Redpanda)	`mail.received`	Decouples ingest from processing. A traffic spike must never drop mail. The **receiver** creates the topic at startup (4 partitions; replication 1 local, 3 production) rather than relying on auto-create, which is off by default on Redpanda and commonly disabled in production — otherwise the failure surfaces as `UNKNOWN_TOPIC_OR_PARTITION` on live mail instead of at boot. Partitions cap how many workers can consume in parallel: easy to raise later, not to lower.

8. Data model

The columns below are the truth. They exist in three places, in this order of authority:

Where	What it is
`backend/migrations/versions/`	The SQL that actually built the database, plus the refcount trigger and the `pgcrypto` extension
`backend/src/nimbus/models.py`	The same tables as SQLAlchemy models. Every query in the app goes through these — there is no raw SQL anywhere in `src/nimbus/`
This section	The human-readable summary

`--autogenerate` diffs the models against the live database, so drift is detectable: the generated migration must be empty. It **cannot** see triggers or extensions, which is why the initial migration is hand-written and must never be regenerated from the models — doing so would drop the refcount trigger silently. See `DATABASE.md`.

```
-- Tenancy
reseller(id, name, api_key_hash, webhook_url)
domain(id, reseller_id, name, verified, catch_all_mailbox_id)
mailbox(id, domain_id, local_part, password_hash, quota_bytes, plan,
        is_shared,        -- true → ONE copy here; members read it (§9.2)
        used_bytes)       -- logical usage, maintained in the delivery/delete txn
alias(id, domain_id, local_part, target_mailbox_id)
shared_mailbox_member(shared_mailbox_id, member_mailbox_id)
forwarding_rule(id, mailbox_id, forward_to_addr, keep_copy)

-- Storage engine  (the heart of the project)
-- Dedup is scoped per reseller, not global — see §12, cross-tenant leak.
-- That is why reseller_id is part of the key and not just the hash.
blob(reseller_id, hash CHAR(64),   -- SHA-256 of the whole attachment
     size_bytes,
     chunk_count,
     refcount,                     -- how many mailbox_message rows reach this blob
     created_at,                   -- when the bytes were STORED. Never moves.
     refcount_zeroed_at,           -- when the blob BECAME GARBAGE. NULL iff refcount > 0.
                                   -- This is the clock GC's grace period measures, and
                                   -- created_at is NOT — see §9.7.
     PRIMARY KEY (reseller_id, hash))

chunk(reseller_id, hash CHAR(64),  -- SHA-256 of this 4 MB piece
      size_bytes,
      s3_key,
      refcount,                    -- how many blob_chunk rows point here
      PRIMARY KEY (reseller_id, hash))

blob_chunk(reseller_id, blob_hash, seq, chunk_hash)   -- ordered chunk map

-- Messages
message(id, reseller_id, raw_s3_key, message_id_header, in_reply_to,
        subject, from_addr, sent_at, received_at, size_bytes, thread_id,
        body_text,          -- what the mail SAYS. Extracted at delivery, capped at 1 MB.
        body_html)          -- Re-parsing per read costs 187 MB/message — see §16.

message_attachment(message_id, blob_hash, reseller_id, filename, content_type, size_bytes)

-- Fan-out: one message, many mailboxes
mailbox_message(mailbox_id, message_id, folder, is_read,
                snooze_until,      -- snoozed IS `snooze_until > now()`. No flag: block H
                                   -- dropped is_snoozed, since two columns for one state
                                   -- can disagree and nothing runs to expire a snooze.
                received_at,       -- the INBOX sort key, set from the arrival time,
```

```
                                -- never left to now() or Kafka lag reorders inboxes
                PRIMARY KEY (mailbox_id, message_id))

-- Search
-- One row per DELIVERED COPY, not per message — same key as mailbox_message.
-- The foreign key points at mailbox_message, so deleting one copy takes its index
-- row with it. Without that, a deleted message stays findable forever (§9.7).
message_index(mailbox_id, message_id, tsv TSVECTOR,
              PRIMARY KEY (mailbox_id, message_id),
              FOREIGN KEY (mailbox_id, message_id)
                  REFERENCES mailbox_message ON DELETE CASCADE)
-- GIN (mailbox_id, tsv), not GIN (tsv). Needs btree_gin. See §9.4 for the measurement.

-- Provisioning
provision_order(id, reseller_id, idempotency_key UNIQUE,
                status,
                payload JSONB,     -- what was asked for
                result  JSONB,     -- what was created; replayed on a retry
                created_at)

-- Exactly-once processing (Kafka delivers at-least-once)
processed_event(event_key TEXT PRIMARY KEY,   -- the raw message's s3_key
                message_id, processed_at)
```

Key idea: `mailbox_message` is the fan-out table. 40 recipients = 40 tiny rows, 1 `message` row, 1 `blob` row. Bytes are stored once.

Refcount rules (get these wrong and GC deletes live data):

- `blob.refcount` counts **`mailbox_message` rows**, not messages. `+N` when a message fans out to N mailboxes, `-1` per row deleted. Counting messages would under-count and free a blob two mailboxes still read.
- `chunk.refcount` counts **`blob_chunk` rows**. `+1` when a blob first references the chunk, `-1` when that blob is deleted. A chunk shared by two blobs survives either one going away.

Who moves each counter — this is not obvious and getting it wrong is silent:

Counter	Up	Down
`blob.refcount`	block **D**, on fan-out. It moves by the number of rows actually written, never by the recipient count — two addresses can resolve to one mailbox, and a replay finds rows already there.	the `mailbox_message` delete trigger
`chunk.refcount`	block **C**, once per `blob_chunk` row it creates	block **G**, when it deletes a blob
`mailbox.used_bytes`	block **D**, on delivery	the same trigger

`chunk.refcount` is the trap. **No trigger touches it** — the one on `mailbox_message` only moves `blob.refcount` and `mailbox.used_bytes`. It is pure application bookkeeping, so if the worker does not increment it, nothing ever does, every chunk sits at zero, and GC deletes the entire chunk store while it is still in use.

A blob created by block C sits at `refcount = 0` until D points at it — but GC never sees that state, because **C and D run inside one transaction** (`worker/pipeline.py`), and an uncommitted row is invisible to the sweep's snapshot. The grace period is not what makes this safe, and an earlier version of this paragraph said it was; §9.7 corrects that in detail. It is worth being precise, because the two claims disagreed inside one document and the wrong one implied an operational rule nobody needs to follow.

What genuinely protects a blob that is committed at `refcount = 0` — one delivered to zero mailboxes, say — is that `refcount_zeroed_at` carries `DEFAULT now()`, so it is collected one grace period after it was stored rather than never. See §9.7. - `thread_id` is set at insert: look up `in_reply_to` → parent's `thread_id`, else start a new one. (Full JWZ is §16.)

Delete behaviour — see `DATABASE.md` §9 for the reasoning:

- `blob`, `chunk` and `message` use `ON DELETE RESTRICT` on `reseller_id`, not cascade. A cascade would delete every row holding an `s3_key`, leaving the objects in S3 with nothing left that knows they exist. Deleting a reseller is a staged operation: empty mailboxes → refcounts reach 0 → GC removes the S3 objects → then the reseller.
- A **trigger on `mailbox_message` DELETE** is the single place `blob.refcount` and `mailbox.used_bytes` go down. Cascade deletes (deprovisioning a mailbox or domain) happen inside the database where application code never runs, so doing this in code would leave refcounts permanently too high and GC would free nothing.
- Foreign keys are indexed selectively, not universally. The two that matter most are `message_attachment(reseller_id, blob_hash)` and `blob_chunk(reseller_id, chunk_hash)` — both are on the GC delete path, which runs constantly.

---

9. Key flows

9.1 Inbound mail

1. Sending server opens SMTP connection to the Go receiver.
2. At `RCPT TO`, the receiver checks the **full address** — not just the domain — against a Redis set of valid addresses (mailboxes, aliases, catch-all domains), written by the API at provisioning time. Unknown address → `550 No such user` **while the sender is still connected**. This is the only point where a rejection is possible; see 9.2.
3. Receiver **streams** the raw message to S3 while it arrives. Memory stays flat.
4. Receiver publishes `mail.received` to Kafka. Connection closes.
5. Worker consumes the event. Steps below run in **one Postgres transaction** whose first statement is `INSERT INTO processed_event (event_key = s3_key) ON CONFLICT DO NOTHING RETURNING event_key`. Nothing returned means this event was already handled, so the worker stops and acks. Without this

guard, a Kafka redelivery double-inserts `mailbox_message` rows and double-increments `blob.refcount` — silent data corruption, not a crash.

`ON CONFLICT DO NOTHING` rather than catching the unique violation: in Postgres any error aborts the whole transaction we are about to do the real work in. It also handles the concurrent case for free — a second worker on the same event blocks on the row lock until the first commits or rolls back. - Resolve each recipient's domain to its **reseller** — `reseller_id` is half of every blob and chunk key, so nothing can be named without it (block C) - Parse MIME (use stdlib, do **not** hand-write a parser) (block C) - For each attachment: chunk → SHA-256 each chunk → hash the whole blob (block C) - Ask Postgres "do we have this blob hash for this reseller?" → if yes, upload nothing and store nothing but the link row (block C) - If no: upload missing chunks to S3, then insert `chunk`, `blob` and `blob_chunk` rows and increment `chunk.refcount` (block C) - Insert 1 `message` row per reseller + its `message_attachment` rows (block C) - Resolve routing (see 9.2) (block D) - Insert N `mailbox_message` rows (block D) - Increment `blob.refcount` by N, and each recipient's `mailbox.used_bytes` (block D) - Build the search index entry (block F)

9.1a What the receiver actually answers (implemented, block B)

Every reply code below is a decision about whether someone's mail survives. A `5xx` where a `4xx` belongs destroys the message permanently.

Situation	Code	Why that one
Recipient is a known mailbox, alias, or catch-all domain	`250`	
Recipient is unknown	`550 5.1.1`	Permanent, and correct — the mailbox genuinely does not exist
Redis unreachable at `RCPT TO`	**`451 4.3.0`**	Never `550`. A `550` tells the sender the mailbox does not exist; it bounces and the mail is gone. `451` means "retry shortly", and it will. Our outage must not delete someone's email.
Message over `MAX_MESSAGE_MB`	`552`	Permanent, and immediate — the message is the same size on every retry, so `451` would make the sender re-transmit 40 MB for days before giving up
S3 upload or Kafka publish fails	`451 4.3.0`	Our fault, so keep the message alive in the sender's queue

Size cap: 40 MB, not 25. A 25 MB attachment is base64-encoded on the wire, which inflates binary by about a third — measured at **35.4 MB** for a 25 MB file. A literal 25 MB cap would reject exactly the case §2 advertises. Advertised in EHLO as `SIZE 41943040`. Configurable via `MAX_MESSAGE_MB`.

The null sender is accepted. `MAIL FROM:<>` is not a malformed envelope — it is the null sender that every bounce, delivery-status notification and auto-reply uses. Rejecting it means never receiving a single bounce.

Recipients are deduplicated by the receiver. A sender may legally repeat `RCPT TO` for the same address. Recording it twice would fan out two `mailbox_message` rows and increment `blob.refcount` twice for one email — and the extra count never returns to zero, so GC could never free that blob. The worker can rely on the list being unique.

The event shape (the worker is written against this):

```
{
  "s3_key":      "raw/2026/08/09/3ce2147f768f5826.eml",
  "from":        "ceo@globex.com",
  "recipients":  ["sujal@acme.com"],
  "size_bytes":  35872886,
  "received_at": "2026-08-09T22:35:28.772Z",
  "remote_addr": "203.0.113.9:58274"
}
```

`s3_key` doubles as `processed_event.event_key`, which is what makes a Kafka replay harmless. `from` is empty for the null sender.

Order: store to S3, then publish. Publish-first would let the worker receive an event pointing at an object that does not exist. Store-first can only leave an unreferenced object in S3 if the publish fails — cheap, sweepable, and nobody loses mail. Fail towards keeping the message.

Connection ceiling: 100 concurrent (`MAX_CONNECTIONS`), enforced by `netutil.LimitListener` around the plain listener.

There is no other limit on this port. Nobody authenticates to an inbound MX, so anything on the internet can open a socket and hold it for the full five-minute read timeout. Without a cap that is a one-line denial of service: open sockets until the process runs out of file descriptors, and no real mail gets through.

The number comes from memory, not from a guess. A connection that is actively streaming holds `PartSize x (Concurrency+1)` = 10 MB of S3 upload buffer — measured at **14 MB** each (six concurrent 35 MB messages, 84 MB RSS). An idle connection holds almost nothing, so this is a worst case, not a typical one:

Cap	If every connection streams at once	Verdict
50	~700 MB	safe on a shared 2 GiB box
100	**~1.4 GB**	**needs the receiver to have that memory to itself**
1000	~14 GB	nothing we plan to run

100 also clears the §13 ingest target with room: 500 messages/min is 8.3 per second, so 100 slots allow a 12-second average message before the cap is the limit.

> ⚠️ **This contradicts §14 as written.** §14 puts *all* app containers plus a self-hosted Redpanda on one `t3.small` — 2 GiB total. The receiver alone at its cap wants 1.4 GB of that. Either the receiver gets its own instance, or the deploy sets `MAX_CONNECTIONS=50`. Recorded here rather than silently picking a smaller default, because the right fix is a deployment decision in block K.

Trade-off: `LimitListener` makes connection 101 wait in the TCP backlog instead of being told "421 too busy". A rejecting limiter is politer, but sending servers retry for days regardless, and the wait costs five lines against a hand-written accept loop. Revisit if queue depth ever shows up as sender timeouts.

9.1b What the worker actually stores (implemented, block C)

Block C is the dedup engine: it opens the raw message, splits out the attachments, chunks and fingerprints them, and stores only what is genuinely new. It stops before deciding whose inbox anything lands in.

One email can belong to two customers. Recipients at `acme.com` and `globex.com` may sit under different resellers, and dedup is per-reseller. So:

Thing	How many
`processed_event` rows	**1**, always. The raw message's `s3_key` is the primary key.
`message` rows	**one per distinct reseller** among the accepted recipients
`blob` / `chunk` rows	one set per reseller — byte-identical attachments are stored twice on purpose (§12)

`processed_event.message_id` is a single nullable column and cannot name two messages, so it stays **NULL** when more than one reseller is involved. That is a known gap and it costs nothing: the replay guard is the primary key on `event_key`, which never weakens.

Chunk keys are `{reseller_id}/{hash[:2]}/{hash}.bin`. Both parts matter. The `reseller_id` prefix keeps two customers' identical files at different addresses — a shared path would reintroduce the cross-tenant leak the composite primary key exists to prevent. The rest is derived from the content, never random, which is what makes a crash between the upload and the commit self-healing: the retry writes identical bytes to the identical address. Unlike the receiver's random `.eml` keys, these cannot orphan.

Order: upload to S3 → insert rows → commit → ack Kafka. Same direction as §9.1a — fail towards keeping mail. All uploads finish before any row is inserted, so no row lock is held across a 4 MB HTTP request; interleaving them makes a second worker touching the same chunk wait out the whole upload run.

The Redis blob-existence cache from §9.1 was dropped. Nothing invalidates it when GC deletes a blob, so a stale hit would claim bytes exist after they are gone. It replaces one primary-key lookup in a transaction we are already inside — roughly 0.3 ms — with a consistency hazard. The `blob` table is the only source of truth.

Threading is scoped by reseller and indexed. A reply's conversation is found by matching `In-Reply-To` against `message.message_id_header`. `Message-ID` is written by the *sender*, so this needs `message_thread_lookup_idx` on `(reseller_id, message_id_header)`: the index for speed — it is a scan of every message otherwise — and the `reseller_id` leading column for isolation. It is also not unique, so the oldest match wins, or a replay could thread the same reply differently twice.

⚠️ **Residual risk:** the reseller filter stops cross-tenant threading, not cross-customer threading *within* one reseller. A sender who guesses a `Message-ID` can have their mail appear inside an existing conversation. That is phishing amplification, not disclosure — thread reads are still scoped by `mailbox_message`. Revisit if threads ever become shareable.

Failure handling. Transient errors — a Postgres deadlock between two workers storing blobs that share chunks, an S3 timeout, a broker blip — are retried 3 times with a widening gap. After that the worker exits **without acking**, so nothing is lost and the mail is redelivered on restart. Skipping the message would lose somebody's email silently, which is the one outcome worth stopping over. No dead-letter topic yet.

Sender-supplied strings are stripped of NUL bytes. RFC 5322 forbids `0x00` in a header, but a sender can put one there and Python's parser passes it through, while Postgres `TEXT` cannot hold it at all. Left alone, one `Subject: a<NUL>b` from anyone on the internet would roll back the transaction, leave the offset uncommitted, and have that message redelivered forever — a remote, unauthenticated halt of a whole partition.

Memory is NOT flat, unlike the receiver. Measured with `tracemalloc` on a 25 MB attachment (33.8 MB on the wire):

	Peak	Held during the database and S3 work
Per message	**187 MB**	**25 MB**

The peak is the stdlib MIME parse and base64 decode, which materialise whole parts — there is no streaming decode in the stdlib, and hand-writing one is exactly what §15 forbids. The held figure is what matters when several workers run at once; the peak is a brief spike per message. **Practical ceiling: about two workers per 2 GiB box** at these attachment sizes. Stated plainly because it is a weaker guarantee than block B's flat 40 MB, and pretending otherwise would size a deployment wrong.

9.2 Routing resolution (Chain of Responsibility)

For each recipient address the receiver **already accepted**, in order — first match wins:

1. Is it an **alias**? → resolve to target mailbox
2. Is it a **mailbox**? → itself
3. Does the domain have a **catch-all**? → deliver there
4. Otherwise → **drop and log**

Alias before mailbox: an alias is an explicit redirect, and a same-named mailbox would otherwise silently win. Nothing stops both existing — they are unique constraints on two different tables — so provisioning should refuse to create an alias over a live mailbox.

Resolution returns a SET of mailbox ids. `sales@` aliased to Alice plus `alice@` directly is two recipients and ONE delivery. Two rows would put the message in her inbox twice and count `blob.refcount` twice, and that second count never comes back down, so GC could never free the blob.

Shared mailboxes are not expanded — changed at v1.0, this section used to say "expand to all members". A shared mailbox receives **one** `mailbox_message` row; `shared_mailbox_member` says who may READ it, which is what `DATABASE.md` always described that table as. Consequences, all deliberate:

	One copy (what we do)	A copy per member (what this said before)
Rows for a 3-member team	1	3
`blob.refcount`	+1	+3
Quota charged	once	to all three
Read state	shared — Alice opens it, Bob sees it handled	private per member

Shared read state is the point of a team inbox: `support@` needs "somebody has this", not three people answering the same mail. The cost lands on block E, whose inbox query must union the mailboxes you own with the ones you are a member of.

Forwarding rules are not consulted. This section used to say "forward (and keep a copy if `keep_copy`)". Forwarding needs an outbound SMTP client, which §4 rules out of v1, and honouring `keep_copy = false` without one would **delete the only copy and send nothing**. Nothing writes `forwarding_rule` either, so the table is inert. The endpoint that creates a rule and the SMTP client that honours it must arrive in the same change.

⚠️ **Shared mailboxes have no on-ramp yet.** Nothing in the codebase sets `mailbox.is_shared` or writes a `shared_mailbox_member` row — provisioning creates ordinary mailboxes only. Delivery and the read path both handle them correctly and are tested, but the rows can currently only be created by hand. The member-management endpoints belong with the reseller API; until they exist this feature is **inert**, the same way `forwarding_rule` is. Recorded so the docs do not describe it as working.

Every resolved mailbox is re-checked against the reseller before delivery. `alias.target_mailbox_id` and `domain.catch_all_mailbox_id` are plain foreign keys to `mailbox.id`, so the schema permits either to point into a different tenant. Nothing can create that pointer today, but the failure would be silent — reseller B's user reads reseller A's mail while every counter stays consistent. One query at the end of resolution drops anything that is not ours.

Step 4 is not a bounce, and cannot be. The address was accepted at `RCPT TO` and the SMTP connection is long closed by the time this code runs — there is nobody left to say `550` to. Sending a real bounce message would mean outbound mail, an explicit non-goal (§4). In practice this branch only fires on a race: the mailbox was deleted between `RCPT TO` and processing. Log it and move on.

9.3 Reading a message with an attachment

1. Client requests `GET /messages/{id}/attachments/{blob_hash}`
2. One query authorises AND fetches metadata: it joins `message_attachment` to a `mailbox_message` row in the caller's **readable set**. No readable row, no attachment.
3. API looks up the ordered chunk list from `blob_chunk`
4. API streams the needed slices from S3 in order to the client
5. Supports HTTP **range requests**, so the client can resume or seek

Memory used: one chunk slice at a time. Never the whole file — a 40 MB attachment costs the same as a 40 KB one, which is the entire reason it was chunked on the way in.

Slices are pushed down to S3, not fetched-then-cut. A 1 KB browser range out of a 4 MB chunk moves 1 KB across the network. Fetching the chunk and slicing locally would move 4 MB and is the difference between a video seeking and re-downloading.

Naming the file by `(message_id, blob_hash)` is what makes it safe. Blobs are shared between messages and mailboxes by design, so a hash alone identifies nothing you are entitled to. Authorisation runs off `message_id`; the hash only says *which* file within a message you can already read. `message_attachment`'s primary key is `(message_id, blob_hash)`, so pairing a hash you learned elsewhere with a message id you can see matches zero rows.

`ETag: "<blob_hash>"` and `Cache-Control: immutable`. The bytes at a content hash cannot change — that is what content addressing means — so re-opening a thread is a `304`, not another download. The strongest possible validator, for free.

The database connection is released before streaming starts. FastAPI runs a yield-dependency's cleanup *after* the response is fully sent, so without an explicit close the pooled session stays checked out for the whole transfer. At `pool_size=10`, eleven concurrent downloads on slow connections would block every other request in the API — including `/health` and login.

9.4 Search

`GET /v1/search?q=...&limit=&cursor=` — mailbox JWT. **Built and verified (block F).**

The query language. One string, split into structured filters plus the words left over.

Token	Means	Compiles to
`from:VALUE`	sender contains VALUE	`message.from_addr ILIKE '%VALUE%'`
`has:attachment`	carries at least one file	`EXISTS (message_attachment)`
`before:YYYY-MM-DD`	arrived strictly before	`mailbox_message.received_at < DATE`
`after:YYYY-MM-DD`	arrived on or after	`mailbox_message.received_at >= DATE`
anything else	free text	`message_index.tsv @@ websearch_to_tsquery(...)`

It is a flat set of filters, not an AST — an earlier version of this section said AST. Four operators that all combine with `AND` need no tree, and `nimbus/api/search_query.py` is a pure function returning one dataclass. String in, filters out, no database, so it is tested offline with no stack at all.

`parse()` never raises. Anything it does not understand — `to:jane`, `before:banana`, `has:wings` — is left in the free text rather than rejected. A search box that answers `400` because someone typed a word with a colon in it is a worse product than one that searches for what they typed.

`websearch_to_tsquery`, never `to_tsquery`. `to_tsquery('foo &')` raises a syntax error, so an ordinary typo would be a `500`. `websearch_to_tsquery` never raises **and** understands `"exact phrase"`, `-exclude` and `or` — which is why the parser deliberately never touches a quote character. Verified: 8 hostile query strings all answer `200`.

The index is `GIN (mailbox_id, tsv)` on one table — not partitioned per mailbox. This settles the open question in §16. `mailbox_id` has to be a column *inside* the index so Postgres can satisfy the tenant filter during the index scan instead of rechecking it after a join. Measured on a 100k-message mailbox with a second tenant holding 50k rows containing the same word:

Index	Rows read from the index	Time
`GIN (mailbox_id, tsv)`	1,000 — this tenant only	**5.3 ms**
`GIN (tsv)`, as originally specified	51,000, incl. 50,000 from the other tenant	16.5 ms

The gap grows linearly with the number of tenants sharing a common word. That is the "one giant global index" this section always warned about — it was in the schema from the start and nothing had written a row to notice. Needs the `btree_gin` extension, which supplies a GIN operator class for `uuid`.

One index row per delivered copy, keyed `(mailbox_id, message_id)` — the same key as `mailbox_message`. The text is identical for every recipient, so it is stored N times. That is accepted deliberately: it is a few KB of derived words beside a 25 MB attachment, and the dedup engine's savings are in the attachment. See §9.7 for the delete rule.

Results are ordered by arrival, newest first — not by relevance. `ts_rank` cannot be keyset-paginated (there is no stable ordering to compare a cursor against), so ranking would force `OFFSET` and reintroduce the skipped-row bug §10.4 bans for the inbox. In one person's mailbox, recency is most of what relevance means. Pagination reuses block E's cursor exactly — `nimbus/api/cursor.py` is shared by both endpoints so they cannot drift.

Search covers every folder, including trash, and includes snoozed mail — both differ from `GET /v1/messages`, deliberately. The inbox is a view of what needs attention now; search is "find that email". Someone who snoozed a message until Monday and then goes looking for it expects to find it. Every result carries its `folder` so the UI can label a trashed or snoozed hit instead of hiding it.

The tenant filter is `visibility.readable_mailboxes()`, never the JWT's mailbox id. Same rule as §9.3, and the stakes here are the same: a shared mailbox holds ONE copy that its members read, so filtering on the token's own id would make every team inbox permanently unfindable — including to the mailbox itself, which has `password_hash = NULL` and cannot be logged into. Verified live: a member finds the shared mailbox's mail, and a third user in the same domain finds none of it.

9.5 Snooze

Built and verified (block H). It is a predicate, not a job.

```
snoozed  <=>  snooze_until > now()
```

Nothing runs to expire a snooze. Time passes on its own, the predicate stops being true, and the message is back in the inbox. Accuracy is exact rather than bounded by a poll interval — §13's "within 1 second" is beaten by not having a timer at all.

This replaces the five steps this section used to specify, and deletes all of them:

Was specified	Why it is gone
`ZADD snooze_queue <T> <id>`	`mailbox_message.snooze_until` already held the time
A Go worker polling every second	There is nothing for it to do
`is_snoozed = true/false`	Derived state that could disagree with `snooze_until`. **Column dropped** (migration `c81f4e6a29d3`)
"Only one node may fire each timer"	No node fires anything
Lock per shard, or leader election	A problem created by the queue, not by snooze
"Push an unread notification"	There is no notification system, no websocket, and §4 forbids outbound mail. The UI polls; the message simply appears

Why not Redis. §9.1b already dropped the Redis blob-existence cache and the dedup lock in block C, with the reason "a lock adds a store that can go stale". A snooze queue is worse than either: it is not a cache, it is the *only* record that a timer exists. Lose the Redis volume and every pending timer is gone silently — mail hidden for ever, no error anywhere, and nothing to rebuild from. `addresses.py` rebuilds its Redis set from Postgres at every startup precisely because Redis is disposable here. It also leaks: deleting a snoozed message removes the row but leaves the sorted-set entry orphaned, and nothing cleans it up.

The read path does not get slower. Measured on a 100k-message mailbox with 10% snoozed: the stored flag and the computed predicate give identical plans and identical buffer counts (0.110 ms vs 0.103 ms, `Index Scan + Filter`, 4 buffers each). No index was added — a partial index on `(snooze_until)` exists only to serve a sweep, and there is no sweep.

The API. Snooze is per-copy state on `mailbox_message`, exactly like `is_read` and `folder`, so it extends the existing endpoint rather than adding two more:

```
PATCH /v1/messages/{id}?mailbox_id={copy}
  {"snooze_until": "2026-08-11T09:00:00Z"}   snooze
  {"snooze_until": null}                      un-snooze now
GET /v1/messages?folder=inbox&snoozed=true    list the snoozed ones
```

`?snoozed=true` inverts the filter rather than adding a pseudo-folder, so `folder` stays an equality and `mailbox_message_list_idx` is still fully used. Without it a snoozed message would be unreachable except through search.

The timestamp must carry a timezone; a naive one is a 422. Not pedantry — measured on this machine, `2027-01-01T09:00:00` with no offset became `03:30 UTC`, so the mail would have reappeared five and a half hours early, silently. Rejecting it at the boundary turns a wrong answer into an error.

Un-snoozing early needs `{"snooze_until": null}` to be distinguishable from the field being absent, which the usual `if value is not None` idiom cannot do. The handler tests `"snooze_until" in model_fields_set` for this one field only; `is_read` and `folder` keep the simpler idiom because null means

nothing for either.

Threads and search deliberately still show snoozed mail (§9.4), and so does opening a message by id. Snooze hides a message from one list view; it does not make it unfindable, and that is how a user reaches one to un-snooze it.

Not addressed: an un-snoozed message returns to its original `received_at` position, so it reappears part-way down the inbox rather than at the top. Moving it would mean rewriting `received_at`, which reorders the inbox and interacts with the keyset cursor. The design this replaced had exactly the same behaviour.

9.6 Provisioning (reseller API)

`POST /v1/orders` with an `Idempotency-Key` header:

```
{ "domain": "acme.com", "mailboxes": ["sujal", "sales", "support"], "plan": "30GB" }
```

- Idempotency key is stored `UNIQUE`. A retry returns the original result, never double-creates.
- All mailboxes are created in **one transaction** — all or nothing.
- On completion, call the reseller's `webhook_url` with retries and backoff.

A domain we already own is reused, not rejected. A reseller adding a second batch of mailboxes to `acme.com` is ordinary business. The domain is inserted with `ON CONFLICT (name) DO NOTHING RETURNING id`; when that returns nothing, a follow-up `SELECT ... WHERE name = $1 AND reseller_id = $2` decides the difference between "ours already" and "someone else's".

Done this way rather than by catching the unique violation because in Postgres **any** error aborts the whole transaction, and every statement after it still has to run inside that transaction. Catching would mean savepoints for a case that has a plain SQL answer.

What each 409 means:

Situation	Status	Body
Domain owned by another reseller	409	`domain not available`
Mailbox name already exists in this domain	409	`mailbox already exists: bob@acme.com`
Same `Idempotency-Key`, first call finished	201	the original result, `"replayed": true`
Same `Idempotency-Key`, first call still open	409	`...still in progress` — retry shortly

The first row is worded vaguely on purpose. "Registered to another reseller" would let anyone with an API key probe which domains our other tenants own, one guess at a time.

The last row used to fall through to a bare `500`. Nothing was created either way, but a 500 reads as our bug and stops a client retrying; a 409 tells it exactly what to do.

Webhook URLs must be public HTTPS. Before calling one, the host is resolved and every returned address checked with `ipaddress.ip_address(...).is_global`; private, loopback, link-local and reserved ranges are refused and logged.

A webhook URL is a stored string that our own server then fetches. Left unchecked, `https://169.254.169.254/latest/meta-data/` makes the API hand over the EC2 instance's IAM credentials — server-side request forgery, reaching a network the caller cannot. Known gap: the address is checked at resolve time, not at connect time, so a DNS record that flips between the two slips through. Closing that needs a pinned-IP transport and is only worth it once webhook URLs become self-service.

Passwords. The reseller does not supply them. The API generates one temporary password per mailbox, returns it **once** in the response, and stores only the Argon2 hash. A retry with the same idempotency key replays the order result but **not** the passwords — there is nothing to replay, by design. Idempotency promises "nothing was created twice", not "secrets are repeatable".

The Redis address cache — the contract with the SMTP receiver (§9.1 step 2).

Key	Type	Holds
`valid_addresses`	SET	every `local_part@domain`, mailboxes and aliases
`catch_all_domains`	SET	domains with a catch-all mailbox

- Written **after** the provisioning transaction commits, never before. Publishing an address before the commit would make the receiver accept mail for a mailbox that may never exist.
- Rebuilt from Postgres on every API startup, so a crash between commit and cache write heals itself. Redis is a cache here, never the source of truth.
- Swapped atomically (build a temp key, `RENAME` over the live one). A plain delete-then-fill leaves a window where the set is empty and every message is rejected.

9.7 Garbage collection

Built and verified (block G). `uv run python -m nimbus.gc [--dry-run]` — one sweep, then exit. Not a daemon: it is not event-driven, it carries no state between runs, and every phase is a no-op unless the previous one has run, so a crash needs no recovery.

- `blob.refcount` drops when a `mailbox_message` row is deleted.
- GC finds `refcount = 0` blobs that have been garbage longer than a grace period (24h).
- Deleting a blob decrements each chunk's refcount; chunks at zero have their rows removed and their S3 objects deleted — **when the object store accepts the delete.** On a partial failure the rows still go, because the alternative is worse (see "what a failed S3 delete must not do" below), and the sweep exits non-zero so a cron job cannot report success while objects are left unreferenced.

`--dry-run` **performs the real deletes and rolls the whole sweep back at the end.** It cannot work any other way, and the first implementation proved it: it rolled back after each phase, so phase 2 never saw the `message_attachment` rows phase 1 would have released and phase 3 never saw the chunks phase 2 would have zeroed. It reported `0 blobs, 0 chunks` **in every real garbage state** — on the one command an operator runs to decide whether GC is worth scheduling, in a system where nothing schedules GC automatically. Two of its three numbers were structurally always zero and looked like a measurement.

The cost is that a preview holds its row locks for the whole sweep rather than one batch, so a dry run against a large backlog can block delivery while it runs. That is acceptable for a command someone types deliberately, and it is exactly why the real sweep commits per batch instead.

What a failed S3 delete must not do. Phase 3 deletes objects inside the transaction, so a partial failure leaves some keys gone and some not. Rolling back looks tidier and is the dangerous choice: every chunk row survives at `refcount = 0` while the keys that *did* delete are already gone, and the next delivery of those exact bytes finds the row, skips the upload, and raises the refcount on a chunk pointing at nothing. Committing costs money — orphaned objects nobody is tracking. Rolling back costs correctness. Commit, exit non-zero, and let an operator reconcile.

The grace period measures `refcount_zeroed_at`, not `created_at`, and the difference is not cosmetic. `created_at` is when the bytes were first stored and it never moves, so a blob written a year ago whose last reference vanished one second ago would pass `created_at < now() - 24h` on the very next sweep — no wait at all — while a blob written an hour ago is held for 23 more hours for nothing. The protection was strongest exactly where it was least needed. Block G added `blob.refcount_zeroed_at`, maintained by the delete trigger on the way down and cleared by `routing.deliver()` on the way up, with the invariant `refcount_zeroed_at IS NULL` exactly when `refcount > 0`.

What the grace period is actually for — this changed in block G. It is *not* what makes the concurrent-upload race safe. **The foreign key is.** `message_attachment` references `blob` with `NO ACTION`, so its key-share lock either blocks GC's delete until the writer commits (after which `WHERE refcount = 0` matches nothing) or blocks the writer until GC commits (after which the writer fails loudly and Kafka retries the whole event). That holds at *any* grace period, including zero. What the 24 hours actually buy is a window in which a human can notice a refcount that reached zero when it should not have, before the bytes are gone. Worth having — but a different justification from the one this section used to give, and worth stating, or the next reader deletes it as redundant.

Phase 1 has no grace period, deliberately. A `message` row nobody holds a copy of is garbage immediately, and holding it back would only delay the blob behind it. The bytes are what the grace period protects.

Which means the 24 hours buy less than the paragraph above claims, and this is worth being honest about. Phase 1 of the *same sweep* has already deleted the orphan `message` row — subject, sender, body — and cascaded `message_attachment`, which held the filename and content type. So during the 24-hour window an operator has the bytes and no record of what they were or who held them, which is not much of a recovery window. The real recovery path is the **7-day raw `.eml` retention** (§11.2): longer, and it still has the whole message. The grace period is worth keeping because it costs nothing, but it is a last-moment safety margin, not a restore feature.

The cutoff is computed by Postgres, not by the application host. `refcount_zeroed_at` is written by the refcount trigger using the database's `now()`, so comparing it against Python's clock measures the skew between two machines as well as the grace period. At 24 hours that is noise; at the `timedelta(0)` the tests use, a database clock a few seconds ahead makes phase 2 collect nothing at all and look correct doing it.

The S3 client carries explicit timeouts because of this phase. boto3's defaults are a 60-second connect, a 60-second read and legacy retries — about 600 seconds before a call gives up. Phase 3 waits on S3 *inside* its transaction while holding `FOR UPDATE` on up to `BATCH` chunk rows, and any worker storing an attachment that shares one of those chunks needs `FOR KEY SHARE` on the same rows. One object-store hiccup would therefore stall mail delivery for ten minutes, with nothing in the logs connecting the two. `storage.py` now sets 5s connect, 30s read and 3 standard-mode retries.

GC is a TWO-PHASE sweep, and it has to be. Found while verifying block D, proven against a live database: deleting a `refcount = 0` blob directly raises

```
ForeignKeyViolationError: update or delete on table "blob" violates foreign key
constraint "message_attachment_reseller_id_blob_hash_fkey" on table "message_attachment"
```

`message_attachment` references `blob(reseller_id, hash)` with no `ON DELETE` clause, so `NO ACTION`. Nothing in the system deletes a `message` or a `message_attachment` row — `DELETE /messages/{id}` (§10.2) removes the `mailbox_message` row only. So a blob reaches `refcount = 0` and becomes **permanently uncollectable**: every byte the dedup engine saves would be stored forever, and the whole point of blocks C, D and G with it.

The sweep, in order:

Phase	Deletes	Why it must come first
1	`message` rows with **no** `mailbox_message` rows left	cascades `message_attachment`, releasing the FK
2	`blob` rows at `refcount = 0` past the grace period	now legal; decrements each chunk's refcount
3	`chunk` rows at `refcount = 0`	then delete the S3 objects

Phase 2 must precede phase 3 for a second reason nobody had written down. `_store_attachment` short-circuits on the *blob* row: if the blob exists it never looks at chunks at all — no query, no upload, no chance to notice anything missing. So a surviving blob whose chunks had been collected would serve a corrupt attachment silently, and the chunk-level dedup check would never run to catch it.

Three rules the implementation proved necessary. Each is a way to get this wrong.

1. **Phase 1 needs TWO statements, not one.** A single `DELETE FROM message WHERE NOT EXISTS (SELECT 1 FROM mailbox_message ...)` is unsafe. If a concurrent transaction is inserting a `mailbox_message` row, the DELETE blocks on the foreign key's lock — and when it unblocks it **deletes the parent anyway**. Postgres re-evaluates conditions against the row itself (EvalPlanQual), and this row was never updated, only key-share locked, so the `NOT EXISTS` keeps its stale snapshot. The cascade then removes the copy that had just been committed: mail silently gone, and the trigger drops `blob.refcount` and `used_bytes` with it. The fix is `SELECT ... FOR UPDATE SKIP LOCKED`, then a separate `DELETE` which under READ COMMITTED takes a fresh snapshot.
2. **Chunk refcounts move by `count(*)`, never by 1.** One blob can reference the same chunk twice — 8 MB of zeros is two identical 4 MB slices — and block C incremented once per `blob_chunk` row. Decrementing by one strands that chunk at `refcount = 1` for ever, its bytes never reclaimed, and nothing raises.

3. **The S3 delete happens INSIDE the transaction, immediately before COMMIT.** Committing the row delete first and calling S3 after leaves a window: the worker's next transaction finds no chunk row, re-uploads the bytes, and the `DeleteObjects` lands afterwards — a live chunk pointing at nothing. Content addressing does not save it, because the re-upload is byte-identical and the delete still arrives last.

Two invariants future blocks must not break. Neither is enforced by the database.

1. **Nothing may add a `mailbox_message` row to an already-committed `message`.** Phase 1 is safe partly because `routing.deliver()` is the only writer and always runs in the same transaction that created the message. "Restore from trash", "undelete" or "copy to another mailbox" all break this — re-read rule 1 above before adding one.
2. **Nothing may delete a `message` row that still has `mailbox_message` rows.** Deleting a message cascades to `mailbox_message` (firing the refcount trigger) *and* to `message_attachment`, in an order Postgres does not guarantee. When the attachment rows go first the trigger joins to nothing: `blob.refcount` is never decremented and that blob is **permanently invisible to phase 2**, while `used_bytes` stays permanently high. GC phase 1 is safe only because it filters to messages with zero copies — that filter is load-bearing for the refcounts, not just for the foreign key. `DATABASE.md` §9 trap 2, path 4, has the proof.

Known and not addressed in v1: `processed_event` grows by one row per email received for ever, and no phase of the sweep touches it (~10 MB per 100k messages). Rows older than the Kafka topic's retention cannot guard against a replay that is no longer possible, so they are prunable — but it is a table nobody is close to being hurt by, and pruning it is code in the one block where mistakes destroy data. Revisit when the row count matters.

`message_index` **needs no phase of its own, and that is deliberate.** Block F points its foreign key at `mailbox_message(mailbox_id, message_id)` with `ON DELETE CASCADE`, so an index row dies with the copy it describes — whether that copy is removed by a user deleting one message or by phase 1 sweeping an orphan. Verified against the live database in both directions, including the two-level cascade `message → mailbox_message → message_index`, and phase 1 still deletes orphans with no foreign-key violation.

Pointing it at `message` and `mailbox` instead — which is how the table was originally built — would have been silently wrong. `DELETE /v1/messages/{id}` removes one `mailbox_message` row and nothing else, so the index row would survive: a deleted message would keep appearing in search results, and opening one would `404`. Worse in a shared mailbox, where deleting your own copy would leave the team's copy untouched but yours orphaned and still matching.

The FK is deliberately **not** changed to `ON DELETE CASCADE`. That would let a blob delete strip a live message's attachment list — the constraint failing loudly is the thing that proves the ordering is right. Failing loudly beats silently detaching.

This also covers the case block D creates on its own: if every recipient's mailbox disappears between `RCPT TO` and processing, `deliver()` writes zero rows and the message, its attachments and its blob sit at `refcount = 0`, referenced by no inbox. Phase 1 collects it because it has no `mailbox_message` rows.

---

10. API surface and auth

10.1 Two callers, two auth schemes

Caller	Credential	How
Mailbox user (webmail, React UI)	password	`POST /v1/auth/login` → JWT, HS256, 24h, claim `mailbox_id`. No refresh token in v1 — re-login.
Reseller (provisioning)	API key	`Authorization: Bearer <key>`, checked against `reseller.api_key_hash`. Long-lived.

The rule that matters: `mailbox_id` and `reseller_id` come from the token, never from the URL, body or query string. Every query is filtered by them. This is the whole of multi-tenant isolation — one missed filter is a cross-tenant data leak.

Passwords: Argon2id. API keys: random 32 bytes, stored hashed, shown once.

The JWT is checked against the database on every request. A signed token is valid until it expires no matter what happened since, so deleting a mailbox would leave its holder reading mail for up to 24 more hours. In a business email product that is an ex-employee with a live inbox — the exact thing an admin pressing "delete" believes they just stopped.

`current_mailbox` therefore runs one primary-key lookup (`SELECT 1 FROM mailbox WHERE id = $1`) after verifying the signature. Sub-millisecond, and every authenticated endpoint queries Postgres anyway.

Trade-off, stated plainly: the token is no longer verifiable offline. We gave up the main advantage of a stateless JWT to get revocation. A Redis deny-list would keep the offline property, but it needs a second store to stay correct and only pays off once auth checks stop touching Postgres for other reasons.

10.2 Endpoints

All under `/v1`. Mailbox JWT unless marked **[reseller]**.

Method	Path	Does
`POST`	`/auth/login`	`{address, password}` → `{token}`
`GET`	`/messages?folder=&limit=&cursor=&snoozed=`	List a folder, keyset paginated. `snoozed=true` shows the snoozed mail instead of hiding it
`GET`	`/messages/{id}`	One message: body + attachment metadata
`GET`	`/messages/{id}/attachments/{blob_hash}`	Stream the file, supports `Range` (9.3)
`PATCH`	`/messages/{id}?mailbox_id=`	`{is_read, folder, snooze_until}` — read/unread, move, archive, snooze. `snooze_until` must carry a timezone (naive → 422); `null` un-snoozes
`DELETE`	`/messages/{id}?mailbox_id=`	Delete ONE `mailbox_message` row, drop refcounts
`GET`	`/threads/{thread_id}`	Messages in a thread, oldest first
`GET`	`/search?q=&limit=&cursor=`	Query DSL (9.4). Same cursor as `/messages`
`GET`	`/quota`	`{logical_bytes, physical_bytes, quota_bytes}` — the demo. Exact definitions in §11.1; `physical_bytes` must not be summed across mailboxes
`POST`	`/orders` **[reseller]**	Provision, needs `Idempotency-Key` (9.6)
`GET`	`/orders/{id}` **[reseller]**	Order status
`GET`	`/domains/{domain}/mailboxes` **[reseller]**	List what exists
`DELETE`	`/mailboxes/{id}` **[reseller]**	Deprovision

Deliberately absent: anything that sends mail. See §4.

10.3 A message id does not identify a row

`mailbox_message`'s primary key is `(mailbox_id, message_id)`, and one email can reach one reader **twice** — copied to their own address and to a shared mailbox they belong to. Both rows are theirs to read. So:

Endpoint	Behaviour
`GET /messages`	Returns **both** copies, each with its own `mailbox_id`. Same `id`, different inbox.
`PATCH` / `DELETE`	Act on exactly **one** copy. Ambiguous request → `409` naming the count, not a guess.
`GET /threads/{id}`	De-duplicates: a conversation shows the message once.

The `409` is not pedantry. Sweeping up every readable row means marking a personal copy read also marks the team's copy handled — and deleting a personal copy destroys the team's only copy of an email, with no undo and no confirmation.

10.4 What a reader may see

Every read query filters on `visibility.readable_mailboxes()` — the caller's own mailbox plus every shared mailbox they are a member of — and on nothing else. One endpoint that forgets it is a cross-tenant leak.

That function re-checks the reseller. `shared_mailbox_member` is two plain foreign keys to `mailbox.id`, so the schema permits a row joining two different customers, exactly as `alias.target_mailbox_id` does on the delivery side (§9.2). One such row and a user reads another company's inbox while every counter stays consistent and no log fires.

Pagination is keyset on `(received_at, message_id, mailbox_id)`, not `OFFSET`. An inbox receives mail while it is being paged, and `OFFSET` re-counts from the top, so a message arriving between pages is pushed across the boundary and never seen. All three columns are needed: the first two tie when one message lands in two of the reader's mailboxes, and a strict `<` on a tied cursor drops a copy from every subsequent page, permanently.

Not found is `404`, never `403`. A `403` confirms that an id someone guessed is real.

---

11. Quota: logical vs physical

	Meaning	Example
Logical	What the user believes they used	1 GB
Physical	What we actually stored	25 MB

Decision: bill logical. Reasons:

- The user's mental model is "my mailbox holds 1 GB".
- Dedup savings are a business margin, not a customer discount.
- Physical is what we track internally to prove the engine works.

The dashboard shows both — that contrast *is* the demo.

11.1 What the two numbers mean exactly (block G)

`GET /v1/quota` returns three keys and nothing else:

Field	Definition
`logical_bytes`	`mailbox.used_bytes` — the sum of `message.size_bytes` over this mailbox's copies
`physical_bytes`	the sum of `size_bytes` over the **distinct blobs** this mailbox's messages reach
`quota_bytes`	`mailbox.quota_bytes`

`physical_bytes` must never be summed across mailboxes. Three mailboxes sharing one 25 MB attachment each report 25 MB; the reseller stored 25 MB, not 75. The reseller-level figure is a different query (`SUM(size_bytes)` over that reseller's blobs) answering a different question. The per-mailbox number

was chosen over the amortised `size_bytes / refcount` because the amortised one sums correctly but means nothing to a human — "you are storing 0.6 MB of a 25 MB file" is not a sentence anyone acts on.

It is reported per mailbox row, never over the readable set. Quota is a fact about one `mailbox` row — each has its own `quota_bytes` — so summing it across shared mailboxes would add up numbers that belong to different limits. This is the one read endpoint that does not filter on `visibility.readable_mailboxes()`, and that is correct.

Two ways the contrast flatters us. Both must stay stated.

1. `logical_bytes` counts the whole raw message, including base64 expansion (~1.37x the binary size); `physical_bytes` counts only decoded attachment bytes. Measured on one 8 MB attachment: logical 11,479,712 vs physical 8,388,608 — so part of the apparent saving is encoding overhead, not deduplication.
2. It excludes the raw `.eml` archive — see below.

11.2 Raw message retention: 7 days

Measured on the development stack: `nimbus-raw` held **2004 MB across 275 objects** while `nimbus-chunks` held **310 MB across 155**. The raw archive was **6.5x** the store garbage collection was built to reclaim, and nothing ever deleted it.

Every attachment was therefore stored twice — base64 inside the raw message, and deduplicated in binary in the chunk store — and the raw copy deduplicates across nothing: forty separate emails carrying the same deck write forty full copies, because each is a distinct message with its own key. A "50% saved" headline would have described 13% of what was actually on disk.

Decision: an object-store lifecycle rule expires raw messages after 7 days.
`backend/scripts/apply_raw_retention.py`, run once per environment.

- Not garbage collection. Deleting the raw object in GC phase 1 is more precise, but two resellers receiving one email share a single `raw_s3_key`, so it needs an "is anyone else using this key" guard plus a new index on `message(raw_s3_key)` — more code, in the one block where a mistake destroys customer data. The object store expires by age on its own.
- **What we gave up:** after 7 days there is no original message, so no "download the original `.eml`" for older mail and no replay through the worker. Everything the product serves — headers, body, attachments, search — lives in Postgres and the chunk store, and both are permanent. The 7 days are a recovery window, not a correctness boundary.

11.3 Quota is reported, not enforced

Nothing rejects mail for a full mailbox in v1, and that is a decision rather than an omission.

Enforcing at delivery is not available: by the time the worker runs the SMTP connection closed minutes ago, and §4 rules out outbound mail, so there is no one to tell. That leaves delivering anyway or dropping — and dropping destroys someone's email to protect a billing limit, which is the worst outcome a mail system can choose.

Enforcing at `RCPT TO` is where it belongs and is still deferred: it needs `used_bytes` in Redis, a second source of truth for the number the product bills on, going stale exactly when it matters. The receiver also does not know the message size at `RCPT TO` — the `SIZE` extension is advisory and senders lie.

What it costs: a mailbox can exceed its quota without limit. The business protection is billing and the dashboard, not blocking. **Upgrade path (block K):** `452 Insufficient system storage` at `RCPT TO` from a Redis figure refreshed on delivery, accepting a stale read — deferred on staleness grounds, not difficulty.

---

12. Known hard problems

These are the interview talking points. Each has a real answer in the code.

Problem	Approach
Two mailboxes receive the same attachment simultaneously	Content addressing makes the write idempotent, so concurrent uploads of the same bytes are correct, only wasteful. The Redis lock originally planned here was **not built** — `ON CONFLICT DO NOTHING` already makes the loser a no-op, and a lock adds a store that can go stale. Verified with two interleaved transactions: 1 blob, 2 chunk-map rows, refcount 1, no double count.
When is it safe to delete a chunk?	Refcount + grace period sweep
Encryption breaks dedup	Identical files encrypt to different bytes per key. Options: convergent encryption, or shared key at rest. Documented trade-off; v1 encrypts at rest with a shared key.
Cross-tenant dedup leaks information	Upload timing reveals whether a file already exists somewhere in the system (published attack on Dropbox). **Mitigation: dedup scoped per reseller/domain, not globally.**
Receiving 25 MB without using 25 MB of RAM	Stream. `io.Copy` in Go, chunked multipart upload to S3
Fixed vs variable chunking	v1 uses fixed 4 MB. Rolling-hash (content-defined) chunking is the upgrade path if we ever need dedup across *modified* versions of a file.

---

13. Non-functional targets

All six now have numbers. Block J (`backend/scripts/loadtest.py`) measured the first two; the rest were settled by the blocks that built them.

Metric	Target	Measured
Dedup ratio on realistic data	> 60% storage saved	**68.8%** on 10,000 messages, seed 42 (§13.1)
SMTP ingest rate	> 500 messages/min sustained	**500/min held**, peak backlog 6, final 0. Ceiling measured separately at **1,417/min** durable — 2.8x the target, worker-bound (§13.2)
Receiver memory with 25 MB attachments	Flat — no growth with attachment size	212 MB of concurrent mail through 84 MB RSS (block B)
Search p95 (100k-message mailbox)	< 100 ms	**5 ms typical, 73–76 ms worst** (block F)
Snooze accuracy	**exact.** Nothing fires — `snooze_until > now()` is evaluated at read time, so there is no timer to be late (§9.5)	exact by construction
Pending snoozes held	1M — but they are just rows with a timestamp, so there is nothing to degrade	nothing to degrade

13.1 The dedup number is a property of the corpus, not the engine

This is the whole reason the load test publishes its generator. Random attachments dedup at ~0%; a corpus built from repeated files dedups at ~100%. Neither proves anything about the engine. So the corpus is defined in the open, the ratio it implies is computed **before** the run, and the measured number is checked against that prediction.

The mix, per 100 messages, and what each share is defending:

Share	Kind	Why
70	no attachment	Most business mail is text. Published attachment rates run 15–25%, so 30% is deliberately generous — and it tilts the result *towards* dedup. Stated because it is the assumption most open to challenge.
24	shared attachment	Circulated documents dominate real mail: decks, invoices, policies, re-attached forwards. Fan-out follows a Zipf curve, capped at 100 sends per file.
6	unique attachment	Genuinely one-off files exist. Without them the ratio is fiction.

Attachment sizes: 55% at 20–200 KB, 35% at 200 KB–2 MB, 9% at 2–10 MB, 1% at 10–25 MB. Mean ≈ 1.15 MB. Recipients per message: 60% to one mailbox, 30% to 2–5, 10% to 6–40 — mean 3.95. At n = 10,000 that is 39,497 deliveries over 720 distinct files.

Three ratios are published, because any one of them alone lies.

	What it divides	Seed 42
R1 dedup alone	distinct stored bytes ÷ attachment bytes across messages	**68.8%**
R2 dedup + fan-out	distinct stored bytes ÷ attachment bytes across copies	**92.0%**
R3 real disk today	chunks + raw `.eml` ÷ what a naive server writes	**68.4%**
R3 after the §11.2 expiry	chunks only ÷ what a naive server writes	**94.2%**

R2 is the flattering one and most of it is **fan-out**, which any mail server gets free by storing one message and many pointers — it is not what the dedup engine earned. R1 is. R3 is the only figure that survives contact with `df`, and it is far below R2 because the raw `.eml` archive measured **3,635 MB against the chunk store's 824 MB — 4.4x** — and deduplicates against nothing. §11.2 measured 6.5x on a different mix; block J reproduces the phenomenon independently at scale. For the first seven days, most of what Nimbus stores is the thing it does not deduplicate.

The spread is itself the finding. Across seeds 42–46 the same mix yields R1 from **65.7% to 78.3% — a 12.6-point swing**, driven by whether a large file lands at a high fan-out rank. A single published number without that spread beside it would be misleading. The headline is seed 42's figure, not the best of five.

What the corpus cannot show. Attachments are random bytes, so two distinct files share no partial content. Fixed 4 MB chunking therefore reclaims nothing that whole-file hashing would not already catch, and the measured sub-chunk saving is ≈ 0. That is a **floor** for the content-defined chunking named in `dedup.py`'s `# ponytail:` comment, never a ceiling — real documents from one template do share runs that a rolling hash would find. §16's first open question stays open.

13.2 The ingest number measures what was held, not what is possible

500 msg/min is the rate the driver **offered**; the system held it with a peak backlog of 6 messages and a final backlog of zero, at 0.0 s of schedule debt. That proves the target.

The ceiling is a separate run, found by offering far more than the system can take:

```
uv run python scripts/loadtest.py --messages 10000 --rate 6000 --threads 64 --seed 42
```

	Result
Durably stored	**1,417 msg/min** — 2.8x the §13 target
Accepted by the receiver	2,959 msg/min — **a floor, not its ceiling** (see below)
Peak queue depth	5,777 messages
Final queue depth	**0** — the backlog drained completely
Integrity checks	**all 11 passed under saturation**

The worker is the bottleneck, not the receiver. The receiver accepted twice what the worker could store, and it was never itself saturated: the driver reached 102.7 s of schedule debt and reported itself as the constraint, so 2,959 msg/min is the fastest the *driver* could offer, not the fastest the receiver could take.

What 1,417 measures is one Python worker doing MIME parsing, chunking, SHA-256, dedup, fan-out and search indexing.

Nothing broke at 2.8x. The backlog grew to 5,777 and drained to zero with every integrity check passing and no retries. Saturation cost latency, not correctness — which is the behaviour the Kafka spool exists to provide (§7). The dedup figures were byte-identical to the paced run, confirming §13.1's claim that the ratio is a property of the corpus and not of the rate.

What this does not tell us. One worker consumes all four Kafka partitions, so the consumer group could scale to four workers without a repartition — roughly 5,700 msg/min if it scales linearly, which is untested. Beyond four, `mail.received` needs more partitions. That is the next thing to know, and block K's sizing decision (§9.1a) should be made against 1,417, not against 500.

Two distinctions the script is built around, both of which flatter the number if blurred:

- **Durable, not accepted.** A `250` from the receiver means the bytes reached `nimbus-raw` and an event reached Kafka. The mail does not exist to any user until the worker commits its rows. The published figure counts committed `message` rows; the accept rate is printed beside it and labelled as the receiver's number.
- **Steady state, not the whole run.** The first and last 10% are discarded. The start pays for plan caching, bucket metadata and Kafka leadership election; the tail is the worker draining a queue with nothing arriving, which is not a sustained rate at all.

Every number here is a floor. The driver, Postgres, MinIO, Redpanda and the worker all compete for one laptop's CPU and disk. On separate hardware each would be higher.

The search number needs both figures, not the flattering one. 5 ms is a selective word — the common case. The worst realistic query is one matching most of the mailbox (`before:2099-01-01` , five characters), where the GIN index cannot help and Postgres sorts 90,000 rows, spilling to disk: **73–76 ms**, inside the target with little room. Nothing can index that `ORDER BY` , because the index is already spent on the `WHERE` . The ceiling is roughly one mailbox of 100k messages; the upgrade path is an index on `(mailbox_id, received_at DESC)` , deliberately not added because it would cost every delivery a write to serve a query shape that currently passes.

13.3 Reproducing it

```
cd backend
uv run python scripts/loadtest.py --corpus-only            # the mix, no stack needed
uv run python scripts/loadtest.py --messages 10000 --rate 500 --seed 42
```

`--seed` **is** the corpus. Attachment bytes come from `random.Random(seed).randbytes()` , so 10,000 messages describing ~4.5 GB of mail are fully specified by two integers and nothing is generated on disk. Publishing the seed satisfies §13.1's demand that the generator ship with the number.

`backend/tests/unit/test_loadtest_corpus.py` asserts the predicted ratios offline, so the figures above cannot drift away from the code without a test failing.

13.4 What the run actually asserts

Eleven checks, all against the driver's own plan rather than the database's internal consistency — a self-consistent database missing 400 messages passes every internal check and reports a *better* dedup ratio than the truth.

Check	Seed 42
messages stored	10,000
copies delivered	39,497
attachments recorded	3,000
search index rows	39,497
distinct files stored (blobs)	720
blob refcount sum	11,643
chunk refcount sum	811
S3 chunk bytes == chunk table	863,750,806
physical bytes stored	**863,750,806**
logical bytes, per message	**2,766,734,026**
logical bytes, per copy	**10,730,417,228**

The last three exist because a ratio cannot check itself. R1 is `1 - physical / logical`, so an error scaling both sides cancels exactly. If the worker regressed to storing base64-encoded bytes instead of decoded ones — the mistake `mime.attachments()` is written to prevent — then every count, every refcount, the S3-versus-database cross-check and R1 itself would be unchanged, while 37% of the chunk store was wasted and every mailbox's `physical_bytes` on the dashboard was inflated. Only comparing the absolute totals against the corpus prediction catches it. This was found by review *after* the first eight checks were written and the first run had already passed.

Two other checks are worth knowing the reason for:

- **`search index rows == copies`.** Block F writes `message_index` inside a SAVEPOINT and deliberately swallows non-transient failures, so a perfect dedup ratio is entirely compatible with zero searchable mail. Nothing else in the report would notice.
- **`messages stored == N` exactly, not `>= N`.** A `451` retry makes the receiver write a *new* random `raw_s3_key`, so `processed_event` — keyed on that key — does not recognise it as a duplicate. The retry becomes a second message carrying the same attachment: a free dedup hit and one message too many.

The standard run is 10,000 messages, not the 100,000 §15 originally named. 100k costs ~59 GB — 47 GB of it the raw `.eml` archive — and 3.5 hours on a laptop that is also hosting the whole stack, which makes iterating on a failure impossible. The ratio is a property of the mix and the mix is identical at both

sizes, so 10k measures it exactly; what 10k does not measure is anything that only breaks at volume — GIN maintenance at 4M rows, worker memory drift over hours, sustained multi-hour Kafka lag. Same generator, one flag apart: run 100k before block K if a machine with the disk is free.

---

14. Deployment

Development: everything in Docker Compose — Postgres, Redis, Redpanda, MinIO. Free, resets in seconds.

Production (Day 10):

Local	AWS
Postgres	RDS
Redis	ElastiCache
MinIO	S3
Redpanda	**Self-hosted in Docker on EC2** — MSK's floor is ~$100/month, not justified at this scale
—	EC2 `t3.small` for the app containers
—	Route 53 for the MX record

Code stays cloud-agnostic: the same `boto3` client talks to MinIO and S3 unchanged.

Configuration comes from `backend/.env`, read by `nimbus/config.py` — a `pydantic-settings` model with **no working defaults**. Real environment variables override the file, which is how production is configured: no `.env` on the box, everything injected by the platform. `.env.example` is the committed template and holds no real secret.

A driver-less `postgresql://` URL is stored once; `config.py` derives `+asyncpg` for the app and `+psycopg` for Alembic. Writing both into `.env` would let them drift, and two copies of a database URL is how you migrate the wrong database.

Settings that must be set before deploying:

Variable	Why it cannot keep its example value
`JWT_SECRET`	HS256 signs every mailbox token with it. Anyone who guesses it forges a token for **any mailbox in any tenant** — all of §10.1 isolation, gone. Must be ≥ 32 bytes (RFC 7518 §3.2); the API refuses to start below that rather than trusting a comment to be noticed.
`DATABASE_URL`	The example points at local Docker Compose with the password in it
`S3_SECRET_KEY`	The dev default is in `docker-compose.yml`, in the repo
`MAX_CONNECTIONS`	Set to 50 while the receiver shares a `t3.small` — see §9.1a

Inbound TLS — not implemented in v1

`STARTTLS` is not offered on the inbound port, so mail and its envelope arrive in cleartext. Large senders (Google, Microsoft) will still deliver but will mark the hop as unencrypted, and some correspondents' policies refuse plaintext delivery outright.

Deferred rather than dismissed: it needs a certificate story (issuance, renewal, which hostname the cert covers) that only makes sense once the MX record points at a real host. Revisit with block K. Recorded here so it is a decision, not an oversight.

Re-confirmed at v0.7 alongside the other block-B review findings, which were fixed. This one stays deferred because the missing piece is not code — `go-smtp` advertises `STARTTLS` as soon as `Server.TLSConfig` is set, about eight lines — it is a certificate for a hostname that does not exist yet. Writing those eight lines now would mean inventing config for a value we cannot fill in.

⚠️ **Port 25 is blocked by default on new AWS accounts.** Develop on 2525. If real inbound mail is wanted by Day 10, file the AWS unblock request on **Day 1** — it takes several days.

---

15. Build order

Blocks, not days. Nothing is cut. The original 10-day figure was an estimate for one person typing; the order below exists to stop us waiting on things we do not have to wait for.

Block	Task	Needs	Risk
A	Docker Compose, SQL schema, FastAPI skeleton, auth (§10), provisioning API, Redis valid-address set	—	low
B	~~**Go SMTP receiver**~~ **DONE** — listens on 2525, `RCPT TO` check against Redis, streams to S3, creates the topic, publishes to Kafka. Verified: 212 MB of concurrent mail through 84 MB RSS.	A	medium
C	~~**MIME split + chunk + SHA-256 dedup + refcounts**~~ **DONE** — consumes `mail.received`, per-reseller dedup, chunk refcounts, replay guard. Measured: two deliveries of one 10 MB attachment stored once, **50% saved**.	A	**high**
D	~~**Routing chain + fan-out delivery**~~ **DONE** — alias/mailbox/catch-all chain, one row per mailbox, `blob.refcount` and `mailbox.used_bytes` maintained. Verified: 4 recipients → 3 mailboxes → refcount 3 → delete one → 2.	C	**high**
E	~~Read path — inbox list, threading, streaming attachment download with `Range`~~ **DONE** — 6 endpoints, keyset pagination, shared-mailbox visibility, byte-range streaming with `ETag`. Verified: a 5 MB attachment returns byte-identical, and a user in the same domain sees nothing of another's.	D	medium
F	~~Search — query DSL parser + per-mailbox index~~ **DONE** — `GET /v1/search`, 4 operators plus free text, composite `GIN (mailbox_id, tsv)`, index rows written at delivery and removed by a cascade. Verified: 5.3 ms on a 100k-message mailbox, a member finds a shared mailbox's mail, another user in the same domain finds none of it.	D	medium
G	~~Quota (logical vs physical) + garbage collection worker~~ **DONE** — `GET /v1/quota`, three-phase sweep in `nimbus/gc.py` with `--dry-run`. Migration `9c3e5a1d7b42` adds the grace clock the design always assumed existed. Verified live: a sweep leaves a shared attachment byte-identical for the mailbox still holding it, a doubly-referenced chunk reaches 0, no refcount goes negative, `used_bytes` reconciles.	D	**high**
H	~~Snooze — Redis sorted set + Go worker~~ **DONE, and it is none of those things.** Snooze is the predicate `snooze_until > now()`, evaluated at read time. Migration `c81f4e6a29d3` drops `is_snoozed`; `PATCH` gains `snooze_until`; the list gains `?snoozed=`. No Redis, no worker, no poll, no lock, no leader election. Verified: a message snoozed for 3 seconds returned by itself with nothing running.	D	medium
I	~~React UI + savings dashboard~~ **DONE** — Vite + React 19 + TypeScript, 5 runtime dependencies, 6 screens. Sender HTML renders in a sandboxed iframe with a prepended CSP; verified against crafted hostile mail that every vector is blocked by the browser. Storage screen shows logical vs physical with both §11.1 caveats. No compose, anywhere.	E F G H	medium
J	~~Load test — 100k messages, measure everything in §13~~ **DONE** — `scripts/loadtest.py`: a seeded corpus generator, a paced SMTP driver and a measurement pass, in one operator script with no new dependencies. It measures the **two** §13 numbers that were unproven; the other four were settled by the blocks that built them, so "everything in §13" was never this script's job. Standard run is **10,000** messages, not 100k — §13.3 says why. Verified: 68.8% dedup, 500 msg/min held, all 11 integrity checks passing (§13.4).	D	medium
K	AWS deploy, README, architecture diagram	J	low

Critical path: A → B → C → D. Everything interesting hangs off D, so get there first. After D, F, G, H and J are independent of each other.

Start these any time — they need nothing running:

- The search query DSL parser (F). It is a pure function: string in, AST out. Unit-testable with no database, no Docker, no mail.
- The load-test corpus generator (J). Writing it early also forces us to decide what the corpus contains, which §13 says is the whole meaning of the dedup number.
- The React shell and styling (I), against mock JSON.

What does not compress

Two things take real-world time no matter how fast the code is written:

Thing	Time	What to do about it
The load test itself	100k messages at the §13 target of 500/min is **~3.5 hours** of wall clock	Settled: the standard run is 10,000 messages and takes **20 minutes**. §13.3 states what shrinking it costs. 100k stays available behind one flag.
AWS port 25 unblock	AWS takes **several days** to approve the request	File it **before writing any code**. It is a form, it costs nothing, and if it is forgotten there is no real inbound mail on the deployed system. §14 already flags this.

Everything else is just writing code.

Rule: do not hand-write MIME parsing, the SMTP protocol state machine, or IMAP. Use libraries. The value of this project is the storage engine, routing, and search — not re-implementing 1982.

---

16. Open questions (to refine)

- [ ] Fixed 4 MB chunks vs content-defined chunking — is fixed enough to hit >60% dedup on realistic mail? **Half answered by block J, and the half it answered is the less interesting one.** Fixed chunking cleared the target comfortably (68.8%) — but on that corpus every saved byte came from *whole files* being sent to many people, not from chunk-level overlap, because random attachment bytes share no partial content. Measured sub-chunk saving was ≈ 0, which is a floor and not a ceiling. The open question is therefore unchanged and now sharper: **on real documents generated from shared templates, how much does a rolling hash find that whole-file hashing misses?** Answering it needs a corpus of real files, which the synthetic generator deliberately is not.
- [x] **Per-mailbox search: one table with `mailbox_id` leading a composite GIN index.** Not partitions. Benchmarked, not argued: on a 100k-message mailbox with a second tenant holding 50k rows containing the same word, `GIN (mailbox_id, tsv)` read 1,000 rows in 5.3 ms while `GIN (tsv)` read 51,000 — including the other tenant's — in 16.5 ms. It is a structural result, not a close call: a `mailbox_id` filter applied *after* a tsv-only index scan can never beat one applied *inside* it, and the gap widens with every tenant added. Partitioning was not needed to get there. See §9.4.
- [x] **Only attachments are deduplicated, not the body** — but the body IS now stored, which is a change from what this said at v1.0. `message.body_text` and `message.body_html` are filled by the worker, which already has the parsed MIME tree. Reading it live from the raw `.eml` instead was

measured at **187 MB peak per 25 MB message** (block C, tracemalloc: the stdlib parser materialises the whole tree including attachments, and there is no streaming decode). That parse would have run inside an API request, on the endpoint every opened email hits, against §14's shared 2 GiB box. Block F needs the same text to build `message_index.tsv` regardless, so extracting once at delivery is work already owed. `body_text` is capped at 1 MB — Postgres refuses a `tsvector` larger than that, so anything beyond it could never be searched anyway. The bodies are still not *deduplicated*; they are stored per message.

- [x] **Snooze worker: neither — there is no worker.** The question assumed a queue that had to be drained exactly once. Snooze is a predicate evaluated at read time, so nothing fires, nothing needs electing and nothing needs sharding. Closed by block H, §9.5.
- [ ] Do we verify SPF/DMARC in v1, or defer? Cheap to add, good CN talking point.
- [ ] Threading: full JWZ algorithm, or simplified `In-Reply-To` chaining?

---

17. Glossary

Term	Meaning
SMTP	Protocol servers use to hand email to each other
MIME	Format inside an email; lets one email carry text, HTML and files
MX record	DNS entry saying "mail for this domain goes here"
Blob	One whole attachment
Chunk	A 4 MB piece of a blob
SHA-256	Turns any bytes into a fixed 64-char fingerprint
Content-addressed storage	Naming a file by its hash instead of its filename
Dedup / Single-instance storage	Storing identical content only once
Chunk map	The ordered list of chunks that reassemble into a blob
Refcount	How many things currently point at a blob or chunk
Fan-out	One message creating rows in many mailboxes
Idempotency	Doing it twice has the same effect as doing it once
At-least-once	The queue promises delivery, but may deliver the same event twice. The consumer must cope.
JWT	Signed token holding "who you are". No session is stored, but see §10.1 — we still confirm the mailbox exists on every request, because a signature cannot tell you the account was deleted an hour ago
RCPT TO	The SMTP command naming a recipient — the last moment you can refuse a message
Multi-tenancy	Many isolated customers on shared infrastructure
Query DSL	A mini search language like `from:x has:attachment`
Inverted index	Map from word → messages containing it
Sorted set	Redis structure ordered by score; used here as a timer queue
p95	95% of requests were faster than this number